De proyecto base a microservicios con JWT

Lab 04 — Ingeniería de Software III

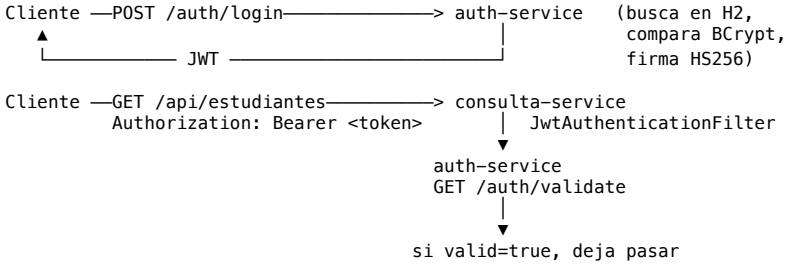
Diego López Camacho — Universidad Industrial de Santander

26 de octubre de 2026

1. Qué se construyó

El laboratorio arranca con un proyecto base de una sola pieza y termina con dos microservicios que se reparten una responsabilidad que antes no existía: decidir quién puede consultar los estudiantes. `auth-service` guarda los usuarios, valida contraseñas y emite tokens; `consulta-service` expone el CRUD de siempre pero ahora exige un token válido, y para saber si lo es le pregunta al primero.

Lo que hace interesante el ejercicio no es que funcione, sino **dónde queda cada decisión**. La contraseña se verifica en un solo sitio. El token se firma en un solo sitio. La comprobación de acceso se hace en un filtro y no repartida por los controladores. Y la dirección donde vive el servicio de autenticación es configuración, no código.



Componente	Puerto	Base de datos
demo-academico (base, Parte 0)	8080	H2 `demoacademico`
auth-service	8081	H2 `authdb`
consulta-service	8082	H2 `demoacademico`

Cada servicio tiene su propia base. No comparten tablas ni se leen entre sí: lo único que cruza es una petición HTTP con un contrato explícito.

Dos notas sobre el entorno. El puerto 8080 de mi equipo está ocupado por otro contenedor ajeno a la materia, así que el proyecto base lo probé en el 8085; en el `application.yaml` queda en 8080 como pide el enunciado. Los puertos 8081 y 8082 sí estaban libres, de modo que los dos microservicios corren exactamente donde se indica, tanto en local como en Docker.

2. Parte 0 — El proyecto base

La arquitectura por capas del curso, construida desde cero con Spring Boot 3.2.5: `controller -> handler -> service -> repository -> model`, más `api` para la respuesta uniforme, `dto` para lo que entra y sale, `exception` para los errores de negocio y `config` para el seeder.

La entidad `Estudiante` creció respecto a laboratorios anteriores: ahora tiene nombre, apellido, email y edad, con el email declarado único tanto en la entidad como verificado en el servicio.

```
$ mvn clean package -DskipTests   ->  demo-academico-0.0.1-SNAPSHOT.jar (52M)
Started DemoAcademicoApplication in 1.963 seconds

  swagger-ui   -> HTTP 200
  v3/api-docs  -> HTTP 200
  h2-console   -> HTTP 302        (redirige al formulario de login de H2)

$ GET /api/estudiantes?page=0&size=2
{"success":true,"message":"OK","data":{"content":[
  {"id":1,"nombre":"Sergio","apellido":"Valladares",
   "email":"estudiante0@demoacademico.edu","edad":18}, ...],
  "totalElements":500,"totalPages":250, ...}}

$ POST /api/estudiantes   (alta válida)
{"success":true,"message":"Estudiante creado","data":{"id":501, ...}}     HTTP 201

$ POST con el mismo email
{"success":false,"message":"Ya existe un estudiante con el email: ..."}   HTTP 400

$ POST con email inválido y edad 0
{"success":false,"message":"Error de validación",
 "data":{"email":"El email no es válido","edad":"La edad mínima es 1"}}   HTTP 400

$ GET /api/estudiantes/999999
{"success":false,"message":"Estudiante no encontrado: 999999"}            HTTP 404
```

El `GlobalExceptionHandler` hace que las tres clases de error salgan con la misma forma que las respuestas correctas. La validación además devuelve el detalle por campo, que es lo que permite a un cliente marcar exactamente qué casilla está mal.

3. Parte 1 — consulta-service, y un efecto colateral

Copiar el proyecto base y cambiarle el nombre fue trivial. Lo interesante vino al agregar las dependencias: `spring-cloud-starter-openfeign` para hablar con el otro servicio y `spring-boot-starter-security` para la protección.

Agregar el starter de Security **sin configurar nada** no deja el proyecto como estaba:

```
Using generated security password: 6efee67f-7e97-4b72-8600-ba58992e4a5d

$ GET /api/estudiantes        -> HTTP 401
$ GET /swagger-ui/index.html  -> HTTP 401
```

Spring Security protege todo por defecto con HTTP Basic y una contraseña generada en cada arranque. Incluso Swagger queda bloqueado. Es un detalle que conviene entender antes de la Parte 4, porque explica por qué el `SecurityConfig` que viene después tiene que abrir explícitamente Swagger, `/v3/api-docs` y la consola H2: no los está "dejando pasar", los está **devolviendo** a un estado que el starter les quitó.

4. Parte 2 — auth-service

4.1 Las contraseñas no se guardan

El seeder crea tres usuarios con `passwordEncoder.encode(...)`. Para comprobar que lo almacenado es realmente un hash y no la contraseña, arranqué el servicio con el log de parámetros de Hibernate en TRACE:

```
1  $2a$10$qCoV64dkZl.mCE/Vb.WMNOah6wU1bzxG8mySa07eF0Sq7eYXWlLkq
2  $2a$10$sY9DFIonoYU3bfzW55myjO8Ya8I1afeDtwvK3BateKeknUkUOmzI6
3  $2a$10$Taqeq2PI0vw7uVg84YqFdefK4W1pUimVEiUQOxvf0k.AeImaGL6.2
```

Sesenta caracteres cada uno, con el prefijo `$2a$10$` que identifica a BCrypt con coste 10. Lo que importa de esta salida es que **los tres son distintos aunque el algoritmo sea el mismo**: BCrypt genera un salt aleatorio por usuario y lo guarda dentro del propio hash. Por eso dos personas con la misma contraseña tendrían hashes diferentes, y por eso no sirve de nada una tabla precalculada.

La consecuencia práctica es que el sistema **no puede recuperar** una contraseña, solo verificarla: `passwordEncoder.matches(password, hash)` vuelve a cifrar lo que llega usando el salt que viene en el hash y compara los resultados.

4.2 El token

```
$ POST /auth/login   admin / Admin2026*
```

```
{"token":"eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoiQURNSU4iLCJzdWIiOiJhZG1pbiIs...","type":"Bearer"}
HTTP 200
```

Decodificando las dos primeras partes del JWT se ve exactamente qué viaja:

```
  header   {"alg": "HS256"}
  payload  {"role": "ADMIN", "sub": "admin", "iat": 1793034754, "exp": 1793038354}
  firma    5x39QEgB3ZvzCFCF2eTT8P9n... (43 chars)
```

El payload **no está cifrado, solo codificado en Base64**: cualquiera puede leer que el usuario es `admin` y su rol `ADMIN`. Lo que impide falsificarlo es la tercera parte, la firma HMAC con la clave del servidor. Por eso un JWT nunca debe llevar datos sensibles, y por eso `iat` y `exp` importan: son la hora de emisión y la de vencimiento, separadas por la hora que configura `jwt.expiration: 3600000`.

```
$ POST /auth/login   contraseña incorrecta   -> "Credenciales inválidas"   HTTP 401
$ POST /auth/login   usuario inexistente     -> "Credenciales inválidas"   HTTP 401
```

Que ambos casos devuelvan el mismo mensaje es correcto desde el punto de vista de seguridad: distinguir "ese usuario no existe" de "la contraseña está mal" le diría a un atacante qué usuarios son válidos.

```
$ GET /auth/validate con token válido
{"valid":true,"role":"ADMIN","username":"admin"}                          HTTP 200

$ GET /auth/validate con prefijo "Bearer " delante
{"valid":true,"role":"ADMIN","username":"admin"}                          HTTP 200

$ GET /auth/validate?token=token-basura
{"valid":false,"message":"Token inválido o expirado"}                     HTTP 401
```

El segundo caso es obra de `TokenUtils.normalize`, que quita el prefijo `Bearer` las veces que haga falta y sin distinguir mayúsculas. Es una tolerancia pequeña que evita el error más común al copiar tokens a mano.

4.3 Un detalle del SecurityConfig que vale la pena señalar

Probando la validación de entrada encontré un comportamiento que no cuadraba:

```
$ POST /auth/login con username vacío   ->  HTTP 401, cuerpo vacío
```

Debería ser un 400 con el detalle del campo. El log muestra que Spring **sí** resuelve la excepción correctamente:

```
Resolved [org.springframework.web.bind.MethodArgumentNotValidException ...]
```

El problema está después. Spring reenvía la respuesta de error a `/error`, y esa ruta no aparece en la lista de `permitAll` del `SecurityConfig`, así que la atrapa `anyRequest().authenticated()` y la convierte en 401. Lo confirmé pidiendo esa ruta directamente:

```
$ GET /error   ->  HTTP 401
```

El arreglo es agregar `"/error"` a los `requestMatchers` permitidos. Dejé el archivo tal como lo da el enunciado para no desviarme de la guía, pero el efecto es que **cualquier error no manejado de auth-service llega al cliente como un 401 vacío** en lugar de su código real, lo que complica el diagnóstico. Conviene saberlo antes de perder tiempo buscando el problema en el token.

5. Parte 3 — OpenFeign

`AuthFeignClient` declara la llamada en cuatro líneas, sin armar la petición HTTP a mano:

```
@FeignClient(name = "authClient", url = "${services.auth.url}")
public interface AuthFeignClient {
    @GetMapping("/auth/validate")
    TokenValidationResponse validate(@RequestParam("token") String token);
}
```

La decisión que más rinde después está en la `url`: sale de `services.auth.url`, no está escrita en el código. Eso es lo que permite que en la Parte 5 baste una variable de entorno para pasar de `localhost` a `http://auth-service:8081` sin recompilar nada.

`TokenValidationService` envuelve el cliente y reduce todo a un `boolean`:

```
public boolean isValid(String rawToken) {
    try {
        String token = TokenUtils.normalize(rawToken);
        if (token == null) return false;
        TokenValidationResponse response = authFeignClient.validate(token);
        return response != null && Boolean.TRUE.equals(response.getValid());
    } catch (Exception e) {
        return false;
    }
}
```

El `catch` merece un comentario, porque es una decisión de diseño y no un descuido. Cualquier fallo —auth-service caído, puerto equivocado, el 401 que el propio servicio devuelve ante un token malo— termina en `false`. La alternativa sería distinguir "token inválido" de "no pude preguntar", pero para el filtro ambas cosas llevan a lo mismo: **si no se puede confirmar que el acceso es legítimo, se deniega**. Más adelante lo comprobé tumbando auth-service a propósito.

6. Parte 4 — El filtro

6.1 Por qué un filtro y no una comprobación en cada endpoint

`JwtAuthenticationFilter` extiende `OncePerRequestFilter` y se registra antes del filtro de usuario y contraseña. La alternativa sería validar el token al principio de cada método del controlador, y eso tiene dos problemas: se repite en cada endpoint nuevo, y basta que alguien olvide la línea para abrir un hueco **sin que nada falle visiblemente**. Centralizarlo hace que la regla por defecto sea proteger, y que las excepciones tengan que declararse explícitamente.

Las excepciones están en `shouldNotFilter`: todo lo que empiece por `/swagger-ui`, `/v3/api-docs` o `/h2-console`, más el `POST /api/estudiantes` exacto. Esa última merece una mención: **crear estudiantes no exige token** aunque la ruta esté bajo el prefijo protegido. Lo comprobé y es así:

```
$ POST /api/estudiantes sin token
{"success":true,"message":"Estudiante creado","data":{"id":501, ...}}      HTTP 201
```

Es lo que pide el enunciado, probablemente para poder cargar datos sin autenticarse, pero en un sistema real sería al revés: escribir suele requerir más permisos que leer, no menos.

El `SecurityConfig` de consulta-service deja `anyRequest().permitAll()`. Es decir, **Spring Security no bloquea nada por sí mismo**: toda la protección la hace el filtro. Es coherente con el diseño, porque la autoridad sobre los tokens está en el otro servicio.

6.2 El guion de pruebas

```
===== 1) Login exitoso — POST /auth/login  admin / Admin2026* =====
{"token":"eyJhbGciOiJIUzI1NiJ9...","type":"Bearer"}                        HTTP 200

===== 2) Login fallido — contraseña incorrecta =====
Credenciales inválidas                                                     HTTP 401

===== 3) Validar token manualmente — GET /auth/validate =====
{"valid":true,"role":"ADMIN","username":"admin"}                           HTTP 200

===== 4) GET /api/estudiantes CON token válido =====
{"success":true,"message":"OK","data":{"content":[
  {"id":1,"nombre":"Eduardo","apellido":"Hinojosa", ...}],
  "totalElements":500, ...}}                                               HTTP 200

===== 5) GET /api/estudiantes SIN autorizar =====
Token no enviado o formato inválido                                        HTTP 401

===== 6) GET /api/estudiantes con token INVÁLIDO =====
Token inválido o expirado                                                  HTTP 401
```

Los dos mensajes del filtro distinguen entre no haber mandado nada y haber mandado algo que no sirve, que para quien está probando es información útil.

6.3 El botón Authorize

```
$ securitySchemes en /v3/api-docs
{"bearerAuth": {"type": "http", "name": "bearerAuth",
                "scheme": "bearer", "bearerFormat": "JWT"}}
  security global: [{"bearerAuth": []}]
```

Declarar el esquema como `http` / `bearer` es lo que hace que **Swagger agregue el prefijo `Bearer` por su cuenta**. Por eso en el campo de Authorize se pega el token pelado. Si uno escribe `Bearer` a mano, el header sale con el prefijo repetido; en este proyecto `TokenUtils` lo tolera porque quita el prefijo en bucle, pero no deja de ser un error.

También comprobé que el header funciona sin prefijo alguno:

```
$ Authorization: <token>   (sin "Bearer")   ->  HTTP 200
```

6.4 Dos comprobaciones que agregué

La llamada entre servicios ocurre de verdad. Para no darlo por supuesto, encendí el log de accesos de auth-service, hice login y dos consultas a consulta-service, y miré qué le había llegado al primero:

```
  POST /auth/login -> 200
  GET /auth/validate?token=eyJhbGciOiJIUzI1NiJ9...DOCENTE... -> 200
  GET /auth/validate?token=basura -> 401
```

Las dos líneas de `/auth/validate` yo nunca las pedí: desde fuera solo llamé a `/auth/login` y a `/api/estudiantes`. Las originó consulta-service a través de Feign.

Qué pasa si auth-service se cae. Con un token válido y recién emitido, tumbé auth-service y repetí la consulta:

```
  (auth-service detenido; consulta-service sigue arriba, Swagger responde 200)
$ GET /api/estudiantes con el MISMO token válido
Token inválido o expirado                                        HTTP 401
```

El token no cambió y sigue siendo legítimo, pero ya no hay quien lo confirme, así que el `catch` devuelve `false` y el filtro deniega. Es el comportamiento correcto, y también deja clara la contrapartida de este diseño: **validar contra el servicio en cada petición hace que auth-service sea un punto único de falla para todo el sistema**. La alternativa sería que consulta-service verificara la firma por su cuenta con la clave compartida, lo que lo haría independiente a costa de repartir el secreto entre los servicios.

7. Parte 5 — Docker

Los Dockerfiles solo copian el jar; no compilan. Eso obliga a ejecutar `mvn clean package` antes de construir, y es el enfoque que en el Trabajo en Clase 04 llamamos "compilación separada".

```
$ docker compose build
Successfully tagged microservicios-demoacademico-auth-service:latest
Successfully tagged microservicios-demoacademico-consulta-service:latest

$ docker compose ps
NAME              STATUS          PORTS
auth-service      Up 20 seconds   0.0.0.0:8081->8081/tcp
consulta-service  Up 20 seconds   0.0.0.0:8082->8082/tcp

$ docker images
microservicios-demoacademico-consulta-service  886MB
microservicios-demoacademico-auth-service      864MB
```

Las imágenes pesan bastante porque parten de `eclipse-temurin:21-jdk` completo. Con el multi-stage sobre una base alpine del Trabajo en Clase 04 bajarían a unos 660 MB; mantengo el enfoque del enunciado, pero el dato queda anotado.

7.1 El punto importante del Compose

```
consulta-service:
  depends_on:
    - auth-service
  environment:
    SERVICES_AUTH_URL: http://auth-service:8081
```

Dentro de un contenedor, `localhost` es **el propio contenedor**, no la máquina ni el otro servicio. Si consulta-service conservara `http://localhost:8081` buscaría auth-service dentro de sí mismo y fallaría siempre. La variable de entorno sobrescribe `services.auth.url` sin tocar el código, que es exactamente para lo que se dejó parametrizada en la Parte 3.

Que funcione se apoya en dos cosas que verifiqué:

```
$ docker exec consulta-service env | grep SERVICES_AUTH_URL
  SERVICES_AUTH_URL=http://auth-service:8081

$ docker exec consulta-service getent hosts auth-service
  172.24.0.2      auth-service
```

La red `micro-net` trae un DNS interno que resuelve el nombre del servicio. Y hay que usar el **puerto interno** 8081, no el publicado: la publicación con `ports:` es para llegar desde el host, mientras que entre contenedores se habla directamente.

Sobre `depends_on`, lo mismo que ya habíamos visto en el Trabajo en Clase 03: solo ordena el arranque, no espera a que auth-service esté listo. Aquí no causa problemas porque la primera validación ocurre cuando alguien llama a la API, mucho después de que ambos terminaron de arrancar, pero no es una garantía.

7.2 El mismo guion, contra los contenedores

```
--- 1) Login exitoso (estudiante1) ---  HTTP 200, token emitido
--- 2) Login fallido ---                HTTP 401
--- 3) Validar token ---                {"valid":true,"role":"ESTUDIANTE",
                                         "username":"estudiante1"}   HTTP 200
--- 4) GET /api/estudiantes CON token --- {"success":true,"message":"OK", ...} HTTP 200
--- 5) GET /api/estudiantes SIN token --- Token no enviado o formato inválido  HTTP 401
--- 6) con token inválido ---             Token inválido o expirado            HTTP 401
```

Resultados idénticos a los de la ejecución local. Que no cambie nada al pasar de dos procesos en la misma máquina a dos contenedores en una red de Docker es justamente lo que se busca: lo único que cambió fue una variable de entorno.

Un detalle menor: Compose avisa de que `version: '3.9'` está obsoleto y lo ignora. Lo dejé porque el enunciado lo pide; no afecta al funcionamiento.

8. Checklist final

Proyecto base

- Arquitectura controller -> handler -> service -> repository -> model.
- DTOs de creación, actualización y salida.
- `ApiResponse` y `ResponseBuilder`.
- `GlobalExceptionHandler` con los cuatro casos.
- H2 y Swagger funcionando.
- CRUD de estudiantes probado, incluidos los errores controlados.

auth-service

- Entidad `AuthUser` con username único, rol y hash.
- `AuthUserRepository`, `AuthDataSeeder` con los tres usuarios.
- `PasswordEncoder` con BCrypt, verificado en el log: hashes `$2a$10$` distintos.
- `JwtUtil` firmando HS256 con el rol como claim, y `TokenUtils`.
- `/auth/login` y `/auth/validate` probados en sus casos buenos y malos.

consulta-service

- OpenFeign habilitado con `@EnableFeignClients`.
- `AuthFeignClient` (`name = "authClient"`) con la url parametrizada.
- `TokenValidationService` devolviendo `boolean` y tratando el fallo como inválido.
- `OpenApiConfig` con el esquema `bearerAuth`.
- `TokenUtils`, `JwtAuthenticationFilter` y `SecurityConfig`.
- `GET /api/estudiantes` exige token válido, comprobado en los tres escenarios.

Swagger

- El botón Authorize aparece: `securitySchemes.bearerAuth` está en `/v3/api-docs`.
- El token se pega sin el prefijo `Bearer`, que Swagger agrega solo.
- El endpoint protegido responde 200 después de autorizar.

Docker

- Los dos proyectos compilan y generan su jar.
- Los dos tienen Dockerfile.
- `docker compose up -d` levanta ambos y el guion de pruebas pasa igual.

9. Conclusión

Lo que más me quedó de este laboratorio es que la seguridad acabó siendo, sobre todo, un problema de **dónde poner cada decisión**. La contraseña se compara en un solo lugar y nunca se guarda; el token se firma en un solo lugar y viaja sin secretos dentro; la comprobación de acceso ocurre en un filtro y no repartida por los controladores, de modo que lo que queda sin proteger hay que declararlo a propósito. Nada de eso es más código que la alternativa: es el mismo código puesto donde se puede razonar sobre él.

Las dos pruebas que agregué por mi cuenta fueron las que más me enseñaron, y las dos consistieron en romper algo. Mirar el log de accesos de auth-service me confirmó que la llamada entre servicios existe de verdad y no la estaba dando por supuesta. Y tumbar auth-service con un token perfectamente válido en la mano dejó ver la contrapartida del diseño: validar contra el servicio en cada petición concentra toda la autoridad en un sitio, lo que es bueno para la coherencia y malo para la disponibilidad, porque convierte a auth-service en un punto único de falla.

También encontré dos cosas que el enunciado no advierte y que conviene tener presentes. Agregar el starter de Spring Security a consulta-service bloqueó hasta Swagger antes de configurar nada, lo que explica por qué el `SecurityConfig` posterior tiene que reabrir esas rutas. Y en auth-service, al no estar `/error` en la lista de rutas permitidas, cualquier error de validación llega al cliente como un 401 vacío en vez de su código real: un detalle de una línea que puede costar bastante tiempo de diagnóstico si uno se empeña en buscar el problema en el token.

Por último, el paso a Docker confirmó algo que ya habíamos visto en los trabajos de clase y que aquí se volvió concreto: lo único que hubo que cambiar para pasar de dos procesos locales a dos contenedores fue una variable de entorno. Eso solo fue posible porque la dirección de auth-service se dejó como configuración desde el principio. Si hubiera estado escrita en el código, el mismo cambio habría significado recompilar y reconstruir las dos imágenes.